

HƯỚNG DẪN CÀI ĐẶT WSL TRÊN WINDOWS

Bước 1: Kiểm tra phiên bản Windows hỗ trợ

WSL yêu cầu:

- **Windows 10 Version 2004 trở lên** (Build 19041+)
- **Windows 11** (tất cả phiên bản đều hỗ trợ)

Để kiểm tra phiên bản:

- Nhấn Win + R → gõ winver → Enter

Bước 2: Mở PowerShell với quyền Administrator

- Tìm PowerShell, nhấn chuột phải → Run as administrator

Bước 3: Cài đặt WSL bằng lệnh

Gõ lệnh sau vào PowerShell:

```
wsl -install -d Ubuntu-22.04
```

Lệnh này sẽ tự động:

- Cài WSL
- Cài Ubuntu mặc định
- Kích hoạt các tính năng cần thiết

☞ Sau khi cài xong, bạn cần **khởi động lại máy**.

Bước 4: Khởi chạy Ubuntu (lần đầu)

Sau khi máy khởi động lại:

- Mở **Start Menu** → gõ **Ubuntu** → chọn
- Ubuntu sẽ khởi chạy và yêu cầu bạn:
 - Tạo user
 - Tạo password cho user

Bước 5: Kiểm tra và sử dụng

Mở Terminal WSL (Ubuntu), bạn có thể chạy lệnh Linux như:

```
sudo apt update  
sudo apt install git
```

HƯỚNG DẪN CÀI ĐẶT XV6 TRÊN WSL

Bước 1: Di chuyển đến ổ D và tạo thư mục chứa mã nguồn

WSL cho phép truy cập các ổ đĩa Windows thông qua đường dẫn /mnt.

Thao tác:

```
cd /mnt/d
mkdir HDH
cd HDH
```

Lúc này bạn đang ở D:\HDH của Windows thông qua WSL (đường dẫn /mnt/d/HDH).

Bước 2: Cài đặt các công cụ cần thiết

```
sudo apt-get update && sudo apt-get upgrade
```

```
sudo apt-get install git build-essential gdb-multiarch qemu-system-
misc gcc-riscv64-linux-gnu binutils-riscv64-linux-gnu
```

Tham khảo thêm: <https://pdos.csail.mit.edu/6.1810/2024/tools.html>

Bước 3: Tải mã nguồn xv6 về thư mục HDH

```
git clone git://g.csail.mit.edu/xv6-labs-2024
cd xv6-labs-2024
```

Bước 4: Biên dịch xv6

```
make
```

Bước 5: Chạy xv6 bằng QEMU

```
make qemu
```

Bước 6: Dọn dẹp (nếu cần)

```
make clean
```

HƯỚNG DẪN VIẾT CHƯƠNG TRÌNH TRÊN XV6

Một số syscall có sẵn trong XV6

Syscall	Mục đích
fork()	Tạo tiến trình con
wait()	Chờ tiến trình con kết thúc
getpid()	Lấy PID hiện tại
sleep(t)	Ngủ trong t clock ticks (~1 tick = 10ms)
sbrk(n)	Cấp phát thêm n bytes bộ nhớ
exec(name, args)	Chạy chương trình khác
kill(pid)	Kết thúc tiến trình theo PID
read(fd, buf, n)	Đọc từ file descriptor
write(fd, buf, n)	Ghi ra file descriptor
open/close()	Mở/đóng file

Ví dụ cụ thể: Viết chương trình sleep (gọi syscall sleep của hệ thống)

◆ Bước 1: Tạo file chương trình sleep

Tạo một file mới tên `sleeptest.c` trong thư mục `user/`:

File: user/sleeptest.c

```
#include "user/user.h"

int main() {
    printf("Sleeping for 100 ticks...\n");
    sleep(100); // ngủ khoảng 1 giây
    printf("Woke up!\n");
    exit(0);
}
```

◆ Bước 2: Thêm vào Makefile

Mở file `Makefile` ở thư mục gốc `xv6`, tìm dòng bắt đầu với `UPROGS=` và thêm vào:

```
$U/_sleeptest\
```

Ví dụ:

```
UPROGS=\
    $U/_wc\
    $U/_zombie\
```

```
$U/_sleeptest\
```

◆ Bước 3: Biên dịch và chạy

```
make clean
```

```
make qemu
```

◆ Bước 4: Chạy chương trình trong xv6

Khi boot xong và hiện dấu nhắc \$, gõ:

```
sleeptest
```

Kết quả mong đợi:

```
Sleeping for 100 ticks...
```

```
Woke up!
```

Giải thích Quá trình biên dịch

1. Biên dịch kernel

- Các file `.c` và `.S` trong thư mục `kernel/` được biên dịch thành `.o`:
`kernel/*.c --> kernel/*.o`
- Dùng `riscv64-unknown-elf-ld` để liên kết các `.o` thành file thực thi `kernel/kernel`.

2. Biên dịch chương trình người dùng (user programs)

- Mỗi file `.c` trong `user/` được biên dịch thành `.o`.
- Dùng `user/user.ld` để link các file `.o` với thư viện người dùng (`ulib.o`, `usys.o`, `umalloc.o`, ...) → tạo các file thực thi ELF trong `user/`, ví dụ:
`user/sleeptest.c` → `user/_sleeptest`

3. Tạo hệ thống file ảo `fs.img`

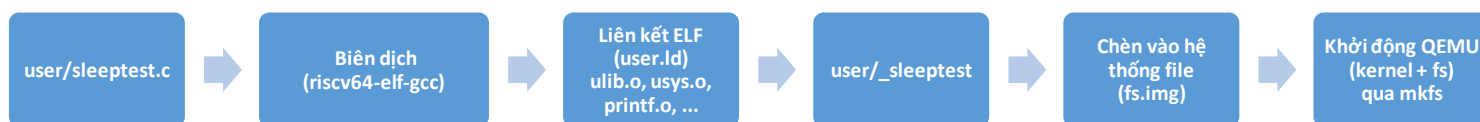
- Gọi `mkfs/mkfs` để tạo `fs.img`, một image chứa toàn bộ các chương trình người dùng như `_init`, `_sh`, `_sleeptest`,...
- `fs.img` sẽ được mount làm file system khi QEMU chạy.

4. Khởi động QEMU

- QEMU giả lập máy RISC-V và nạp `kernel/kernel` làm kernel chính.
- Nó mount `fs.img` làm ổ đĩa chính.
- Kernel gọi `init`, `init` gọi `sh`.

5. Thực thi chương trình

- Tại shell, bạn gõ `sleeptest`.
- Shell gọi `exec("sleeptest")`, kernel đọc file từ `fs.img`, nạp vào bộ nhớ và chạy.



CÁC BƯỚC THÊM MỘT SYSTEM CALL MỚI VÀO XV6.

Giả sử bạn muốn tạo một system call tên là `hello(name)`, dưới đây là chi tiết từng bước:

1) Thêm định nghĩa trong `user.h`:

```
int hello(char *name); // In ra "Hello, <name>"
```

2) Viết hàm xử lý system call trong `sysproc.c`:

```
uint64
sys_hello(void) {
    char *name;
    if (argstr(0, &name) < 0)
        return -1;
    printf("Hello, %s\n", name);
    return 0;
}
```

3) Liên kết trong `syscall.c`:

```
extern uint64 sys_hello(void);
static int (*syscalls[])(void) = {
    ...
    [SYSCALL_NUMBER] = sys_hello,
};
```

4) Cập nhật số hiệu syscall trong `syscall.h`:

```
#define SYS_hello 22
```

5) Thêm entry point trong `usys.p`:

```
entry("hello");
```

6) Kiểm tra trong chương trình người dùng `testhello.c`:

```
#include "kernel/types.h"
#include "user/user.h"

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: hello <name>\n");
        exit(1);
    }
    hello(argv[1]);
    exit(0);
}
```

7) Thêm vào `make file`:

```
$U/_testhello\
```

KHỐI QUẢN LÝ TIẾN TRÌNH

1. struct proc là gì?

`struct proc` là cấu trúc dữ liệu trung tâm đại diện cho một tiến trình trong xv6. Mỗi tiến trình tương ứng với một biến `struct proc`, được lưu trong mảng `proc[NPROC]`.

2. Các thành phần chính trong struct proc

Thành phần	Ý nghĩa
<code>int pid</code>	Mã định danh tiến trình
<code>enum procstate state</code>	Trạng thái tiến trình (RUNNING, SLEEPING, v.v.)
<code>char name[16]</code>	Tên tiến trình (dùng để debug)
<code>void *chan</code>	Nếu khác NULL, tiến trình đang <code>sleep()</code> chờ trên chan
<code>int killed</code>	Đánh dấu tiến trình đã bị yêu cầu hủy
<code>int xstate</code>	Mã trạng thái trả về khi tiến trình <code>exit()</code>
<code>struct proc *parent</code>	Con trỏ đến tiến trình cha
<code>uint64 sz</code>	Kích thước bộ nhớ tiến trình (bytes)
<code>pagetable_t pagetable</code>	Bảng trang ánh xạ không gian người dùng
<code>uint64 kstack</code>	Địa chỉ ảo của kernel stack
<code>struct trapframe *trapframe</code>	Lưu trạng thái thanh ghi khi trap
<code>struct context context</code>	Dùng để lưu trạng thái chuyển ngữ cảnh
<code>struct file *ofile[NOFILE]</code>	Danh sách các tập tin đang mở
<code>struct inode *cwd</code>	Thư mục làm việc hiện tại

3. Các trạng thái của tiến trình (enum procstate)

Trạng thái	Mô tả
UNUSED	Slot chưa được sử dụng
USED	Slot đã cấp phát nhưng chưa chạy
SLEEPING	Tiến trình đang chờ sự kiện
RUNNABLE	Sẵn sàng để scheduler chạy
RUNNING	Đang chạy trên CPU
ZOMBIE	Đã kết thúc, chờ tiến trình cha <code>wait()</code>

4. Vòng đời tiến trình

```

fork() →
  USED → RUNNABLE → RUNNING
        ↘ SLEEPING ↗
        ZOMBIE → UNUSED

```

Tiến trình bắt đầu từ USED, được scheduler chọn để chạy (RUNNING), có thể bị sleep (SLEEPING), sau đó quay lại RUNNABLE, và cuối cùng thành ZOMBIE khi `exit()`.

5. Đồng bộ hóa khi truy cập proc

- `p->lock` được sử dụng khi truy cập hầu hết các trường (ví dụ: `state`, `chan`, `killed`)
- `wait_lock` được dùng riêng để bảo vệ `parent`, đảm bảo an toàn khi tiến trình cha-con tương tác (`exit()`/`wait()`)

6. Cơ chế sleep-wakeup và chan

- `sleep(chan, lock)` → đưa tiến trình vào trạng thái SLEEPING và gán `p->chan = chan`
- `wakeup(chan)` → duyệt qua các tiến trình, nếu tiến trình nào đang SLEEPING trên đúng `chan`, thì đánh thức (RUNNABLE)

`chan` là một con trỏ tùy ý, chỉ dùng để phân biệt lý do tiến trình chờ.

7. Vai trò của trapframe và context

- `trapframe` → chứa toàn bộ thanh ghi user khi trap vào kernel (do syscall, ngắt,...)
- `context` → chứa các thanh ghi callee-saved, dùng khi `swtch()` để chuyển tiến trình

8. Môi liên hệ với CPU

- Mỗi CPU có một biến `struct cpu`:
 - `cpu->proc` → tiến trình đang chạy trên CPU đó
 - `cpu->context` → trạng thái trước khi nhảy vào scheduler

GIỚI THIỆU VỀ FORK TRONG XV6

`fork()` là một hệ thống gọi (system call) trong Unix/Linux, dùng để tạo ra một tiến trình mới, gọi là **tiến trình con**. Khi `fork()` được gọi, hệ điều hành sẽ tạo ra một bản sao của tiến trình cha, và tiến trình con sẽ bắt đầu thực thi từ ngay sau câu lệnh `fork()`.

Khi `fork()` được gọi:

- Hệ điều hành tạo ra một bản sao là tiến trình con của tiến trình cha, trong đó bao gồm cả bảng mô tả tệp.
- Cả tiến trình cha và tiến trình con đều có các mô tả tệp tương tự, nhưng mỗi tiến trình sẽ độc lập khi thực hiện các thao tác trên các mô tả tệp đó.

Khi một tiến trình gọi `fork()`, cả **tiến trình cha** và **tiến trình con** sẽ **chạy song song** và tiếp tục thực thi đoạn mã ngay sau lệnh `fork()`. Việc tiến trình cha hay tiến trình con chạy trước phụ thuộc vào lịch trình của hệ điều hành. Để đảm bảo rằng tiến trình cha tiếp tục thực thi mà không thực hiện các công việc của tiến trình con, bạn có thể sử dụng **giá trị trả về** của `fork()` để **phân biệt hành động** cho từng tiến trình. `fork()` trả về giá trị khác nhau cho tiến trình cha và tiến trình con:

- Với **tiến trình con**, `fork()` trả về giá trị 0.
- Với **tiến trình cha**, `fork()` trả về **PID của tiến trình con** (một giá trị dương).

Điều này có nghĩa là:

- Khi kiểm tra `fork() == 0`, bạn đang kiểm tra xem đoạn mã đó có đang được thực thi bởi **tiến trình con** hay không. Nếu `fork() == 0` là đúng, có nghĩa là bạn đang ở trong tiến trình con và thực hiện các tác vụ của tiến trình con.
- Nếu `fork()` trả về giá trị dương, bạn đang ở trong **tiến trình cha**, và giá trị đó là PID của tiến trình con mà bạn vừa tạo ra.

GIỚI THIỆU VỀ PIPE TRONG XV6

Trong Xv6, hệ thống `pipe()` được sử dụng để tạo một kênh giao tiếp giữa các tiến trình, cho phép truyền dữ liệu từ tiến trình này sang tiến trình khác. Một **pipe** hoạt động như một cặp tệp (file descriptors) — một cho việc **ghi** vào pipe và một cho việc **đọc** từ pipe.

Cách hoạt động của `pipe()`:

1. Tạo pipe:

- Khi một tiến trình gọi `pipe()`, kernel sẽ tạo ra một mảng gồm hai file descriptors: `fd[0]` và `fd[1]`.
 - `fd[0]`: dành cho việc đọc dữ liệu từ pipe.
 - `fd[1]`: dành cho việc ghi dữ liệu vào pipe.
- Gọi `pipe()` trả về mảng `fd[]` này để tiến trình có thể sử dụng. Nếu gọi thành công, tiến trình sẽ nhận được 2 số nguyên đại diện cho hai đầu của pipe.

2. Truyền dữ liệu qua pipe:

- Tiến trình **ghi** dữ liệu vào `fd[1]`. Dữ liệu được lưu tạm trong một vùng đệm (buffer) của kernel.
- Tiến trình khác (hoặc chính tiến trình đó) **đọc** dữ liệu từ `fd[0]`. Khi tiến trình đọc, dữ liệu được lấy ra từ vùng đệm và xóa khỏi pipe.

3. Tính năng của pipe:

- Pipe là một cấu trúc **đơn hướng**: dữ liệu chỉ đi theo một hướng từ đầu ghi (write) đến đầu đọc (read).
- Pipe trong Xv6 không có tên, nó chỉ tồn tại tạm thời giữa các tiến trình. Nếu không còn tiến trình nào sử dụng pipe, kernel sẽ giải phóng tài nguyên.

Các bước chi tiết khi sử dụng `pipe()`:

1. Gọi `pipe()`:

- Một tiến trình gọi `pipe(fd)` để tạo ra hai file descriptors `fd[0]` (đọc) và `fd[1]` (ghi).
- Ví dụ:

```
int fd[2];
pipe(fd); // fd[0]: đọc, fd[1]: ghi
```

2. Tạo tiến trình con bằng `fork()`:

- Sau khi tạo pipe, tiến trình thường sử dụng `fork()` để tạo ra một tiến trình con. Cả tiến trình cha và con đều kế thừa file descriptors của pipe.
- Sau khi fork, cha và con có thể giao tiếp với nhau qua pipe:
 - Cha có thể ghi dữ liệu vào `fd[1]`, và con đọc từ `fd[0]`, hoặc ngược lại.

3. Đóng đầu không cần thiết:

- Để tránh lỗi, mỗi tiến trình nên đóng đầu không sử dụng của pipe:

- Nếu một tiến trình chỉ ghi dữ liệu vào pipe, nó nên đóng đầu đọc (`fd[0]`).
- Nếu một tiến trình chỉ đọc, nó nên đóng đầu ghi (`fd[1]`).

4. Đọc và ghi dữ liệu:

- Tiến trình cha ghi dữ liệu vào pipe bằng cách gọi `write(fd[1], ...)`.
- Tiến trình con đọc dữ liệu từ pipe bằng cách gọi `read(fd[0], ...)`.

Ví dụ minh họa pipe trong Xv6:

Dưới đây là ví dụ đơn giản về cách sử dụng `pipe()` để giao tiếp giữa tiến trình cha và tiến trình con:

```
#include "kernel/types.h"
#include "user/user.h"

int main() {
    int fd[2];          // Mảng lưu 2 file descriptors
    char buffer[20];

    // Tạo pipe
    pipe(fd);

    // Tạo tiến trình con
    if (fork() == 0) {
        // Tiến trình con
        close(fd[1]);    // Đóng đầu ghi vì tiến trình con chỉ đọc
        read(fd[0], buffer, sizeof(buffer));
        printf("Child received: %s\n", buffer);
        close(fd[0]);
    } else {
        // Tiến trình cha
        close(fd[0]);    // Đóng đầu đọc vì tiến trình cha chỉ ghi
        write(fd[1], "Hello from parent", 18);
        close(fd[1]);
    }

    exit(0);
}
```

Một số điểm quan trọng về pipe trong Xv6:

- **Đồng bộ hóa:** Pipe cung cấp cơ chế đồng bộ đơn giản giữa tiến trình ghi và tiến trình đọc. Nếu không có dữ liệu để đọc, tiến trình đọc sẽ bị chặn (block) cho đến khi có dữ liệu. Ngược lại, nếu bộ đệm của pipe đầy, tiến trình ghi sẽ bị chặn cho đến khi có không gian trống.
- **Đóng pipe:** Khi một đầu của pipe (đọc hoặc ghi) bị đóng và không còn tiến trình nào đang dùng, kernel sẽ trả về lỗi cho các gọi hệ thống `read()` hoặc `write()` tương ứng.

GIỚI THIỆU BẢNG TRANG TRONG XV6

Trong xv6, mỗi tiến trình (kể cả kernel) có một **bảng trang riêng** để ánh xạ **địa chỉ ảo (virtual address)** sang **địa chỉ vật lý (physical address)**. Cơ chế này được xây dựng dựa trên **hệ thống phân trang Sv39** của kiến trúc RISC-V.

1. Sv39 – Phân trang 3 cấp

Địa chỉ ảo 64-bit được chia như sau:

Bit	Vai trò
63–39	Bỏ qua (phải là 0)
38–30	Chỉ số cấp 2
29–21	Chỉ số cấp 1
20–12	Chỉ số cấp 0
11–0	Offset trong trang 4KB

- Mỗi bảng trang có **512 mục** ($512 \times 8 \text{ byte} = 4\text{KB}$).
- Một địa chỉ ảo cần **tối đa 3 lần truy cập bảng trang** để tìm ra địa chỉ vật lý.

2. Cách hoạt động tổng quát

- Khi CPU cần truy cập địa chỉ ảo, nó sẽ dùng bảng trang của tiến trình hiện tại để tra địa chỉ vật lý tương ứng.
- Nếu ánh xạ không tồn tại, sẽ gây lỗi trang (page fault).
- Hệ điều hành quản lý bảng trang bằng cách tạo/sửa đổi các mục (PTE – Page Table Entry) trong RAM.
- Mỗi tiến trình có **bảng trang riêng biệt**, lưu trong `proc->pagetable`.

3. Các tập tin liên quan trong xv6

Tập tin	Vai trò
<code>vm.c</code>	Quản lý bảng trang (tạo, ánh xạ, hủy, cấp phát, copy)
<code>defs.h</code>	Khai báo các hàm quản lý bộ nhớ
<code>riscv.h</code>	Định nghĩa bit trong PTE, macro Sv39
<code>memlayout.h</code>	Sơ đồ vùng nhớ ảo của kernel
<code>proc.h</code>	Mỗi tiến trình có 1 bảng trang <code>pagetable_t pagetable</code>
<code>trampoline.S</code>	Mã chuyển đổi user $\leftrightarrow$ kernel, dùng bảng trang đặc biệt

<code>kernel.ld</code>	Định nghĩa vị trí đoạn mã trampoline, etext,...
------------------------	---

4. Các hàm quan trọng trong vm.c

Hàm	Chức năng
<code>kvmmake()</code>	Tạo bảng trang kernel
<code>uvmcreate()</code>	Tạo bảng trang trống cho tiến trình user
<code>walk()</code>	Duyệt bảng trang 3 cấp để tìm (hoặc tạo) PTE
<code>mappages()</code>	Tạo ánh xạ va $\rightarrow$ pa với quyền truy cập
<code>walkaddr()</code>	Trả về địa chỉ vật lý nếu va được ánh xạ
<code>uvmalloc()</code> / <code>uvmdealloc()</code>	Cấp phát / thu hồi bộ nhớ cho tiến trình
<code>uvmcopy()</code>	Sao chép bộ nhớ từ tiến trình cha sang con
<code>uvmfree()</code>	Giải phóng toàn bộ bộ nhớ tiến trình
<code>copyin()</code> / <code>copyout()</code> / <code>copyinstr()</code>	Chuyển dữ liệu giữa user $\leftrightarrow$ kernel

5. Vai trò của bảng trang trong hệ thống

- **Cách ly bộ nhớ** giữa các tiến trình.
- Bảo vệ kernel khỏi truy cập trái phép từ user.
- Hỗ trợ cấp phát bộ nhớ động.
- Cần thiết cho việc chuyển đổi không gian địa chỉ khi context switch.